

KENTECHTIME

A Graduation-Requirement-Aware Automatic Timetable Recommendation Service for KENTECH Students

Introduction to AI Programming — Final Project Technical Report

Team	EFAI Team 10
Members	Mingi Kang, Wooseong Kwon, Hyundam Park, Hyeongjun Koo
Deployment	https://kentechtime.vercel.app
Repository	https://github.com/EFAI-Team10/KENTECHTIME
Tech Stack	Next.js 14 / React 18 / Supabase (PostgreSQL) / OpenAI API
Date	2026-06-07

1. Introduction

KENTECHTIME is a graduation-requirement-aware automatic timetable recommendation web service for undergraduate students at the Korea Institute of Energy Technology (KENTECH). When a student enters their completed-course history and preferences, the service analyzes the remaining graduation requirements and automatically generates candidate timetables for the upcoming semester as Plan A / B / C. The project aims to experience an AI-assisted ("vibe coding") development workflow, while the team takes full responsibility for problem definition, system design, code validation, debugging, integration, and explanation.

2. Problem Definition

KENTECH graduation requirements are complex: beyond per-category credits, they include EF sub-areas (Math, Physics, Chemistry, DL), EL4/5 upper-level courses, sequential ESP stages, and admission-year-dependent requirements. Every semester, students had to manually cross-reference the requirement sheet and the course-offering site to figure out "which requirement does this course satisfy?" This process is time-consuming and error-prone.

Problem	How the service solves it
Complex requirement calculation	Dashboard visualizing requirements from completed-course history
"What should I take to graduate?"	Auto recommendation using unmet requirements and preferences
Tedious manual timetable assembly	Copy a recommendation, edit in My Timetable, click to swap conflicts
Wanting to edit in natural language	LLM conversational editing ("remove Thursday morning classes")

3. Target Users and User Scenario

Target users: KENTECH undergraduate students with an @kentech.ac.kr Google account.

New-user scenario

- Sign in with an @kentech.ac.kr Google account
- Onboarding: name, student ID, semester index, completed courses, and preferences (track, max credits, last semester, gap minimization, ESP start level)
- Enter the main page → view graduation-requirement status and recommended timetables A/B/C

Returning-user scenario

- Copy a preferred timetable among recommendations A/B/C
- Paste into the My Timetable tab and edit (add/remove, click to swap on conflict)
- Save and confirm → reflected in the course-demand competition tracker
- Optionally edit via AI chat in natural language

4. Service Design Rationale

This section explains why the system is designed the way it is and why each design choice is useful for the target users (KENTECH undergraduates) described in Section 3.

Design decisions and their rationale

Design decision	Why (rationale for the target user)
Recommend from graduation requirements, not raw course search	Students struggle most with "which requirement does this course satisfy?" Encoding the rules (per-area credits, EF sub-areas, EL4/5, ESP stages, admission-year branching) removes the manual cross-referencing that is the real pain point.
LLM parses intent only; deterministic code makes all judgments	A timetable with a time conflict or a credit overflow is useless to a student. By letting the LLM only translate language into a JSON intent and having code verify conflicts/credits/requirements, the service never produces an invalid timetable, which builds trust.
Return three plans (A/B/C) instead of one	Course selection is a personal trade-off (track, morning load, gaps). Offering distinct alternatives lets the student choose rather than accept a single opaque answer.
Visual requirement dashboard (donut + gauges)	Reduces the cognitive load of reading a dense requirement sheet, so a student can see at a glance what is left to graduate.
Preference settings (track, max credits, compact, last semester)	Recommendations must fit each student's situation; preferences personalize the result without requiring manual timetable assembly.
Unified Next.js stack + Supabase + OAuth	A single framework for UI and API simplifies the data flow and JWT auth; Supabase provides a managed PostgreSQL so the small team can focus on domain logic; @kentech.ac.kr OAuth restricts the service to real KENTECH students.

5. Service Features and User Flow

Core features

Feature	Description
Auth / Onboarding	Google OAuth (@kentech.ac.kr) + JWT; semester index auto-maps to grade; completed-course & preference input
Requirement Dashboard	Per-area earned/required credits as donut & gauges; EF sub-areas, EL4/5, ESP stages
Timetable Recommendation	Auto-generates Plan A/B/C from unmet requirements and preferences
My Timetable	Copy, edit, multi-create, save, rename, confirm; click-to-swap conflicting courses
Demand Tracker	Competition ratio vs. capacity based on confirmed timetables
AI Conversational Edit	LLM converts a natural-language request into a JSON intent and edits the timetable
UX	Dark mode + 9 color themes, mobile responsive

Service flow: from user input to AI processing to final output

(A) Recommendation flow. (1) Input — the student provides completed courses (via portal import or manual selection) and preferences (track, max credits, compact, last semester). (2) Processing — the client calls POST /api/schedule/recommend; the server analyzes earned credits per requirement, filters out completed and already-satisfied courses, sorts by grade and preferred track, then assembles conflict-free timetables under the credit cap. (3) Output — three distinct plans (A/B/C) are returned and rendered next to the requirement dashboard.

(B) AI conversational-edit flow. (1) Input — the student types a natural-language request such as "remove Thursday morning classes." (2) AI processing — POST /api/chat sends the message together with the current timetable, offered courses, and requirement rules to gpt-5-mini, which returns a structured JSON intent (action + target codes/days); a second LLM call enriches the reply when course details are requested. (3) Output — deterministic code applies the intent (direct edit for add/remove/replace, or re-running the engine for filter) and returns the updated timetable with a natural-language reply.

(C) Confirm & track flow. The student copies a plan into My Timetable, edits and saves it, then confirms it; confirmed timetables feed the demand tracker, which shows the competition ratio against each course's capacity.

6. System Architecture

The client (Next.js / React) calls Next.js Route Handlers (server API) through an axios-based api-client that injects a Bearer JWT. The server verifies the JWT, then accesses PostgreSQL via Supabase (service role), and runs the recommendation logic (recommender.js), the LLM integration (OpenAI API), and Google OAuth verification.

Data flow (login → data request)

(1) User signs in with Google → (2) the id_token is sent to /api/auth/google → (3) the server verifies the id_token with Google → (4) checks registration in Supabase → (5) issues a JWT (or routes to onboarding if new) → (6) all subsequent API calls attach the Bearer JWT → (7) requireAuth validates it and queries via service role → (8) the result is returned.

Main directories

app/ (pages + api/ Route Handlers) - components/ (Dashboard, Timetable, Chat, Tracker) - lib/server/ (recommender, auth, googleVerify, supabase, parser) - database/ (schema + migrations) - public/bookmarklet.js

7. Implementation Details

Core file: lib/server/recommender.js (timetable recommendation engine)

The entry function generateRecommendations() performs the following steps in order.

Step	Function	Role
1	getCompletedInfo	Aggregate completed courses into earned credits per category
2	getRequiredCourses	Keep only unmet + still-required courses (mandatory / elective)

3	applyIntent	Apply the LLM-parsed intent (add/remove courses, exclude days)
4	sortByGradeAndTrack / bumpEL45	Sort by my grade, preferred track, required EL first
5	buildPlan	Assemble a timetable checking time conflict, credit cap, category cap
6	loop (n=3)	De-prioritize used courses to produce distinct Plans A/B/C

Key design decision: the LLM only "translates" natural language into a JSON intent, while all "judgments" — time conflicts, credit overflow, requirement checks — are performed by deterministic code. This fundamentally prevents invalid timetables caused by LLM hallucination.

Core conflict-check logic (hasTimeConflict)

```
if (a.day === b.day && a.start < b.end && b.start < a.end) return true; // overlap on the same day = conflict
```

LLM integration: app/api/chat/route.js

Together with the user message, the system prompt injects the completed-course history, current timetable, all offered courses, and the KENTECH requirement track rules, then calls gpt-5-mini in JSON mode. After parsing the intent (action: remove / add / replace / filter / chat), if course details are needed it issues a second LLM call to enrich the reply. add/remove/replace edit the current timetable directly, while filter re-runs the recommendation engine.

8. AI-Assisted Development Process

Claude, ChatGPT, and GitHub Copilot were used as development assistants, and the OpenAI API (gpt-5-mini) was used as a runtime LLM feature. AI helped with component/API scaffolding, the Express-to-Next.js migration, schema design, debugging, and documentation drafts. See AI_USAGE.md for representative prompts and the full list of modifications.

Representative cases we modified and validated

- Simple recommender → extended with per-category / EF sub-area / admission-year / ESP stage / EL4-5 logic
- Found AP & exchange-credit misclassification → parser auto-classifies AP→EF, exchange→FR
- LLM hallucination risk → separated intent parsing (LLM) from timetable judgment (code)
- Account-deletion FK violation (500) → fixed by deleting child rows first

9. Testing and Demonstration

- Validated recommendations with real KENTECH course-offering xlsx data (no time conflicts, credit caps respected)
- Verified admission-year EF Math branching (4 cr. before 2025 / 8 cr. from 2025)
- End-to-end check: Google OAuth login → onboarding → recommend → save/confirm → tracker
- Live demo of AI natural-language editing ("remove Thursday morning classes")
- Visual check of mobile responsiveness, dark mode, and 9 color themes

10. Discussion and Future Improvements

Limitations

- Recommendation uses a greedy (priority-ordered) approach and does not guarantee a globally optimal combination
- Course offerings and capacities depend on school-provided files
- LLM calls incur cost and latency and are affected by network conditions

Future directions

- Adopt constraint-satisfaction (CSP) / optimization techniques to improve recommendation quality
- Integrate real-time enrollment data and enhance demand prediction
- Strengthen recommendation rationale (which requirement is satisfied) and learn from user feedback

11. References

- KENTECH 2026 Spring academic handbook (source of graduation-requirement rules)
- School-provided course-offering list (xlsx)
- Official docs of Next.js, React, Zustand, Recharts, Supabase
- OpenAI API documentation (accessed via internal gateway)